1.Pod file

```
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  namespace: dev
spec:
  containers:
     - name:
       image:
       ports:
         - containerPort:
```

Service files

2. Pod file for Service file

```
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  namespace: dev
  labels:
    app: web
spec:
  containers:
     - name:
       image:
       ports:
         - containerPort:
```

3. Service file for ClusterIP

```
apiVersion: v1
kind: Service
metadata:
  name: s1
spec:
  type: ClusterIP
  ports:
    - port:
      targetPort:
  selector:
     app: web
```

4. Service file for Nodeport

```
apiVersion: v1
kind: Service
metadata:
  name: s1
spec:
```

```
  type: NodePort
  ports:
    - port:
      targetPort:
      nodePort: 30000-32767
  selector:
     app: web
```

5. Service file for Loadbalancer

```
apiVersion: v1
kind: Service
metadata:
  name: s1
spec:
  type: LoadBalancer
  ports:
    - port:
      targetPort:
  selector:
     app: web
```

---

6. Replication controller

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: rc
spec:
   replicas: 3
   selector:
      app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
```

7. ReplicaSet

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs
spec:
   replicas: 3
   selector:
      matchLabels:
```

```
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
```

8. Deployment file for Replicaset

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
```

---

9. NameSpace file

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

10. Deployment file for Namespace

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
  namespace: dev
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
```

```
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
```

---

11. ConfigMap file

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cmname
data:
   key1: "value1"
   key2: "value2"
```

12. To Inject variables to pods [complete config file]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
               envFrom:
                 - configMapRef:
                        name: cmname
```

13. To Inject particular variables to pods

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
```

```
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
           app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
               env:
                 - name: keyname
                   valueFrom:
                      configMapKeyRef:
                         name: cmname
                         key: keyname
```

---

14. Secret file

```
apiVersion: v1
kind: Secret
metadata:
    name: s1
type: Opaque
stringData:
     key: value
```

15. To Inject variables to pods [complete secret file]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
           app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
               envFrom:
```

```
                - secretRef:
                      name: secretname
```

16. To Inject particular variables to pods [secret file]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
               env:
                 - name: keyname
                   valueFrom:
                      secretKeyRef:
                         name: secretname
                         key: keyname
```

VOLUMES

17. Deployment file for emptyDir

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name: c1
               image: ubuntu
```

```
               command: ["/bin/bash",'-c',"while true; do echo
hello;sleep 300; done"]
               volumeMounts:
                      - name: vname
                        mountPath: /data/path
             - name: c2
               image: amazonlinux
               command: ["/bin/bash",'-c',"while true; do echo
hello;sleep 300; done"]
               volumeMounts:
                      - name: vname
                        mountPath: /data/path
          volumes:
           - name: vname
             emptyDir: {}
```

18. Deployment file for hostPath

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name: c1
               image: ubuntu
               command: ["/bin/bash",'-c',"while true; do echo
hello;sleep 300; done"]
               volumeMounts:
                      - name: vname
                        mountPath: /data/path
             - name: c2
               image: amazonlinux
               command: ["/bin/bash",'-c',"while true; do echo
hello;sleep 300; done"]
               volumeMounts:
                      - name: vname
                        mountPath: /data/path
          volumes:
           - name: vname
             hostPath:
                 path: /data/vol
```

19. Persistent volume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
    capacity:
         storage: 10Gi
    accessModes:
         - ReadWriteOnce
    awsElasticBlockStore:
         volumeID:
         fsType: ext4
    persistentVolumeReclaimPolicy: Recycle
```

20. Persistent volume claim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc1
spec:
    accessModes:
         - ReadWriteOnce
    resources:
           requests:
             storage: 5Gi
```

21. Deployment file for Persistent volume claim

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name: c1
               image: ubuntu
               command: ["/bin/bash",'-c',"while true; do echo
hello;sleep 300; done"]
               volumeMounts:
                      - name: vname
                        mountPath: /data/path
             - name: c2
```

```
               image: amazonlinux
               command: ["/bin/bash",'-c',"while true; do echo
hello;sleep 300; done"]
               volumeMounts:
                      - name: vname
                        mountPath: /data/path
          volumes:
           - name: vname
             persistentVolumeClaim:
                       claimName: pvc1
```

---

22. Daemon set file

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: d1
spec:
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name: c1
               image: ubuntu
               ports:
                   - containerPort: 8
```

23. ResourceQuota

```
apiVersion: v1
kind: ResourceQuota

metadata:
  name: rq1
  namespace: dev

spec:
  hard:
    pods: "5"
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.cpu: "4"
    limits.memory: 4Gi
    storage: 10Gi
    persistentvolumeclaim: '2'
```

---

24. Job

```
apiVersion: batch/v1
kind: Job
metadata:
  name: job1
spec:
    completions: 1
    template:
       spec:
          containers:
             - name: c1
               image: ubuntu
               command: ["echo", "Hello Kubernetes"]
          restartPolicy: Never/OnFailure
```

25. cronJob

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cj1
spec:
    schedule: "* * * * *"
    jobTemplate:
        spec:
          template:
             spec:
               containers:
                      - name: c1
                        image: ubuntu
                        command: ["echo", "Hello Kubernetes"]
               restartPolicy: Never/OnFailure
```

---

26. Recreate Deployment

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
```

```
                - containerPort:
```

27.Deployment file for rolling updates

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
   strategy:
        type: RollingUpdate
        rollingupdate:
             maxsurge: 1
             maxunavalible: 1
```

28. Blue-green deployment file

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
            env: blue
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
```

29.Service file for blue-green deployment

```
apiVersion: v1
kind: Service
metadata:
  name: s1
spec:
  type: LoadBalancer
  ports:
    - port:
      targetPort:
  selector:
     app: web
     env: blue
```

ROLE-BASED-ACCESS CONTROL(RBAC)

a. USER ACCOUNT

30. Role.yml (user)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: role1
  namespace: dev
rules:
   - apiGroups: ["*"]
     resources: ["pods"]
     verbs: ["get","list","watch"]
```

31. RoleBinding.yml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: rb1
  namespace: dev
subjects:
  - kind: User
    name: RAKSHITHA
    namespace: dev
roleRef:
    kind: Role
    name: role1
    apiGroup: rbac.authorization.k8s.io
```

32.CluserRole.yml (user)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: clusterRole
metadata:
  name: cr1
rules:
   - apiGroups: ["*"]
```

```
     resources: ["pods"]
     verbs: ["get","list","watch"]
```

33.CluserRoleBinding.yml (user)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: crb1
subjects:
  - kind: User
    name: RAKSHITHA
    apiGroup: rbac.authorization.k8s.io
roleRef:
    kind: ClusterRole
    name: cr1
    apiGroup: rbac.authorization.k8s.io
```

---

b.SERVICE ACCOUNT

34. Service account.yml for role

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: s1
  namespace: dev
```

35. Role.yml (service)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: r1
  namespace: dev
rules:
   - apiGroups: ["*"]
     resources: ["pods"]
     verbs: ["get","list","watch"]
```

36. RoleBinding.yml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: rb1
  namespace: dev
subjects:
  - kind: ServiceAccount
    name: s1
    namespace: dev
roleRef:
    kind: Role
```

```
    name: r1
    apiGroup: rbac.authorization.k8s.io
```

37. Deployment file for service account(role)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          serviceAccountName:
          containers:
             - name:
               image:
               ports:
                - containerPort:
```

38.Service account.yml for cluster

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cs1
```

39. CluserRole.yml (service)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cr1
rules:
   - apiGroups: ["*"]
     resources: ["pods"]
     verbs: ["get","list","watch"]
```

40.CluserRoleBinding.yml (service)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: crb1
subjects:
  - kind: ServiceAccount
    name: cs1
    apiGroup: rbac.authorization.k8s.io
```

```
roleRef:
    kind: ClusterRole
    name: cr1
    apiGroup: rbac.authorization.k8s.io
---------------------------------------------------------------------
```

INGRESS

41. Host based routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: i1

spec:
  ingressClassName: igc1
  rules:
  - host: shop.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: shop-service
            port:
              number: 80

  - host: blog.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: blog-service
            port:
              number: 80
```

42. Path Based Routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: i1

spec:
  ingressClassName: ig1
  rules:
  - host: example.com
    http:
      paths:
```

```
      - path: /shop
        pathType: Prefix
        backend:
          service:
            name: shop-service
            port:
              number: 80

      - path: /blog
        pathType: Prefix
        backend:
          service:
            name: blog-service
            port:
              number: 80
```

---

43. Statefull file

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: d1
spec:
   replicas: 3
   selector:
      matchLabels:
          app: web
   template:
      metadata:
         labels:
            app: web
      spec:
          containers:
             - name:
               image:
               ports:
                - containerPort:
          imagePullSecrets:
                 - name:
```
